

Matthew Shoup
Arsam Ahmad
Mridual Madan

Implementing RAG Graphs to Enhance LLM Capability

Abstract

We present a retrieval-augmented deep learning framework for fake news detection that integrates graph-based evidence clustering with large language models to improve verifiability and reduce hallucinations. Given an input claim, the system retrieves semantically relevant evidence from a knowledge base, clusters candidate evidence in a graph structure, and generates a fact-checked decision with supporting justification. Claims and evidence are embedded using Sentence Transformer models, compressed via principal component analysis, and organized into semantically coherent clusters using k-means++ to enable efficient multi-hop retrieval. At inference time, relevant clusters are identified through cosine similarity, candidate evidence is re-ranked using a cross-encoder, and the top evidence is passed to a fine-tuned language model to classify the claim as true or false. We evaluate our approach primarily on the FEVER dataset and analyze the impact of graph-based retrieval. Performance is assessed using standard classification metrics for veracity prediction and clustering quality measures to evaluate the structure of the evidence graph. Our results demonstrate that incorporating graph-based retrieval into a RAG pipeline improves evidence selection and supports more reliable and transparent fake news verification.

Introduction

In the modern age, the amount of computational power and convenience provided by artificial intelligence is remarkable. Large language models (LLMs) extend these capabilities by offering assistance and general knowledge to billions of people every day. However, because these models are designed for general-purpose use across many domains, the answers they produce are not always the most reliable. The rapid proliferation of misinformation and fake news—arising from spam, AI-generated content, and the dishonesty of online sources has made it increasingly difficult for users to determine whether a news article or claim can be trusted. These combined issues highlight a critical need for systems that can verify information and provide clear, evidence-backed explanations rather than unverified text.

A well-known limitation of LLMs is that their responses are often unverified and can lead users in the wrong direction if the output is not grounded in trustworthy sources. Because these models are trained to predict text rather than reason over facts, they can hallucinate information or provide claims that cannot be traced back to reliable evidence. We wanted to tackle these glaring issues by devising a solution that improves the trustworthiness of LLM-based systems while

preserving their expressive and interpretive strengths. It would better serve users to have a model dedicated to a specific task of fake news detection and claim verification, so that responses are more up to date and properly supported by evidence.

To address these challenges, we investigated the feasibility of building a system comparable in structure to real-world LLMs, but explicitly designed to verify news articles and claims. Our approach is based on Retrieval-Augmented Generation (RAG), a framework in which a language model is supported by an external knowledge source that provides relevant factual context at inference time. Rather than relying solely on parametric knowledge stored during training, a RAG system retrieves evidence from a corpus of verified information and uses it to guide the final prediction. This allows the model to ground its outputs in real evidence and mitigates many of the hallucination and ambiguity issues found in standard LLMs.

The architecture we chose to research and implement is a graph-based RAG LLM. The core idea is to cluster verified information into semantically meaningful groups and then retrieve the clusters that most closely relate to the user’s query. Claims and evidence are embedded into a shared semantic vector space using sentence-level embeddings which enables efficient similarity-based retrieval. These embeddings are further organized into clusters using k-means++ to form a graph of related evidence. At inference time, the system identifies the most relevant clusters and searches them to extract the most meaningful supporting facts. These are then ranked using a cross-encoder for improved precision. The selected evidence is finally passed to a fine-tuned LLM, which determines whether the input claim is true or false and generates a justification grounded in the retrieved facts.

This design directly addresses several weaknesses of standard LLMs. If a user’s input is ambiguous or vaguely worded, the RAG system can explore multiple relevant clusters and interpretations before selecting the most appropriate evidence. Because the model is tailored to a specific domain, it already understands the context and meaning of news-related queries, which allows it to interpret the user’s intent with higher confidence. Additionally, by restricting the system to verified information, the generated responses have a much higher probability of being factual. While no automated system can guarantee complete correctness, grounding LLM outputs in retrieved evidence significantly improves reliability and transparency.

Our work builds on established fact-checking pipelines, such as those evaluated on the FEVER dataset, which combine evidence retrieval with claim classification. By integrating these ideas into a RAG-based framework and enhancing retrieval with graph-based clustering and re-ranking, we aim to improve both the quality of retrieved evidence and the clarity of the model’s final decision.

Previous Works

To begin our work, we looked at what was implemented by Yang Et Al. and Nasir Et Al. to research methods of fake news detection and understand quality metrics and processes that we could use in our own implementation. We also looked to the work of Chang Et Al. to understand the pipeline process of graph networks to give us a baseline representation to work with and see the strengths and weaknesses of such a model.

Methodology

We developed a pipeline that finds the best evidence embeddings in the graph and delivers them to the LLM for the model to make an informed decision about the query. This process requires many individual pieces working together in a pipeline. The pipeline can be outlined as follows:

1. Data extraction and Preprocessing: We used an open source API to acquire 100 articles from the date range 1/1/25 to 12/1/25 for training. The articles are then converted to a list of verifiable claims that can be converted into embeddings and stored.
2. Word representations: We convert all of the evidence documents into vector embeddings using a pre-trained Sentence Transformer Model.
3. Graph creation and clustering: We group the embeddings from (1) into coherent clusters using k-means++ clustering. Each cluster becomes a node in the evidence graph.
4. Graph Searching: For a given input claim, we identify the most relevant clusters by comparing the embedding of the input to the closest cluster centroids and search for evidence within these centroids.
5. Classification: We concatenate the evidence with the original input and feed it into a fine tuned language model.

Data Extraction and Pre-Processing

For data extraction, we relied on an open-source news API to collect 100 articles published between January 1, 2025, and December 1, 2025. The selected sources were limited to credible and well-established news organizations to ensure reliability and reduce noise in the dataset. Each article included verified factual information, which allowed us to treat the content as trusted reference material. These verified articles form the foundation of our Retrieval-Augmented Generation (RAG) system, serving as authoritative context that the model can query when evaluating new claims related to events that occurred throughout 2025.

The raw article text was transformed into a structured list of short, verifiable claims. This step was critical because atomic claims are easier to embed, retrieve, and reason over compared to long-form text. Each article was broken down into concise factual statements that represent a

single event or assertion. To perform this transformation efficiently, we used the google-flan-t5-base model, a lightweight yet capable NLP model well-suited for text-to-text tasks. Its lower computational cost made it practical for processing large volumes of news content while maintaining acceptable semantic accuracy.

Model Input

The model input consists of the original user claim combined with contextual evidence retrieved from the RAG system. The RAG acts as a structured dictionary of verified events from 2025, where each entry is stored as an embedding representing a factual claim. When a user submits a query, semantically similar claims are retrieved and passed as context to the fine-tuned LLM. This retrieved context provides the necessary background information for the model to reason about the claim, enabling it to determine whether the input is supported, refuted, or lacks sufficient evidence.

Word Representation

Our evidence knowledge base is constructed using the verified evidence provided by the FEVER dataset, which consists primarily of factual sentences extracted from Wikipedia that are known to either support or refute specific claims. Each evidence sentence is treated as an independent unit of information and stored separately in the database. This design allows the system to reason over individual facts instead of entire documents, making retrieval more precise and flexible when evaluating a user's query.

To represent both the stored evidence and the user's input claim in a form that the model can understand, we convert all text into fixed-length numerical embeddings using a Sentence Transformer model. In our implementation, we use a pretrained SBERT-based encoder (such as all-mpnet-base-v2), which maps each sentence to a 768-dimensional vector. This model has been trained on millions of sentence pairs to capture semantic similarity, meaning that sentences expressing similar ideas are positioned close to one another in the embedding space even if they use different wording. The transformer's attention mechanism assigns greater importance to key words, entities, and negations, allowing the embeddings to preserve nuanced contextual meaning.

Because working directly with 768-dimensional vectors is computationally expensive at scale, we apply dimensionality reduction to improve efficiency. We use Principal Component Analysis (PCA) to compress the embeddings while retaining most of their variance and semantic information. In practice, this reduces each vector from 768 dimensions down to approximately 300 dimensions. After PCA, we apply L2 normalization so that each vector has unit length. This step is important because it ensures that cosine similarity computations during retrieval are consistent and efficient, effectively reducing similarity comparisons to a dot product between normalized vectors.

Finally, each processed evidence embedding is stored along with its associated metadata, including a unique identifier linking it back to the original evidence sentence and placeholders for similarity and relevance scores that are computed at query time. This structured representation allows for fast similarity search, clustering, and re-ranking during inference, enabling the system to efficiently retrieve and evaluate the most relevant evidence when determining whether a user’s claim is true or false.

Graph Creation

Once all evidence embeddings have been generated, we organize them into a graph of clusters to capture the overall semantic structure of the knowledge base. We use the k-means++ clustering algorithm to partition the embedding space into groups of semantically related evidence sentences. The motivation behind clustering is that evidence sentences discussing similar topics or facts are likely to be useful together. If a user’s claim is relevant to one sentence within a cluster, other sentences in the same cluster may also provide meaningful support or refutation.

The k-means++ algorithm improves upon standard k-means by using a smarter initialization strategy for cluster centroids. The first centroid is selected uniformly at random from the embedding space. Each subsequent centroid is then chosen with probability proportional to the squared distance from the nearest existing centroid. This encourages the initial centroids to be well spread across the space, reducing the likelihood of poor local optima. After initialization, the standard k-means iterative procedure is applied: each evidence embedding is assigned to the nearest centroid, the centroid is recomputed as the mean of all embeddings assigned to it, and this process is repeated until convergence.

Formally, clustering produces k clusters denoted as $C_1, C_2, \dots, C_k$, where each cluster C_i is associated with a centroid vector μ_i , defined as the mean of all embedding vectors within that cluster. The value of k is treated as a hyperparameter and is chosen based on the size and diversity of the dataset. In our setting, k is selected such that each cluster represents a reasonably coherent semantic topic, for example, hundreds of clusters over tens of thousands of evidence sentences.

This clustered structure can be viewed as a semantic graph over the evidence space. Each cluster corresponds to a node containing a group of related evidence sentences, and although we do not explicitly construct edges between clusters, the distances between centroid vectors implicitly define relationships in the embedding space. While a more advanced implementation could explicitly connect nearby clusters, our current approach uses the centroids directly for search and retrieval.

Clustering provides several important benefits. First, it acts as a coarse filtering mechanism: instead of searching over the entire evidence corpus for every query, the system can focus on only the most relevant clusters, significantly reducing noise from unrelated information. Second, it improves robustness to ambiguous or multi-faceted claims, as the query may activate multiple clusters corresponding to different interpretations. By pre-grouping evidence into

semantically coherent clusters, the retrieval process becomes more focused and efficient, making it easier to identify relevant evidence within a smaller, more meaningful search space.

Graph Searching

In order to determine which clusters hold the most relevant data to the input query, we had to determine which clusters to search. We represent each cluster as the average embedding across all entries within the cluster. We then take the cosine similarity of each cluster and search the top X (changeable parameter) clusters with the highest cosine similarity score to the input query. Inside each cluster, the top N (changeable parameter) evidence embeddings become candidates for return. The top N evidence claims is a list of claims gathered from all searched clusters that have the highest cosine similarity score in relation to the input query. This is done as a quick scan of the data to pull the best options to then be weighted by the Cross Encoder. This is a word comparison transformer for a more in depth search of the best candidates. This will reshuffle and rerank the top N candidates to only select the top K (changeable parameter, less than or equal to N) to be selected as the final candidates. These are then returned to be used as the supporting evidence examples. We then repeat this whole process num hops (changeable parameter) amount of times. This allows for new top N candidates to be found by revisiting each cluster and pulling new candidates that were not chosen before to have another chance of being selected and reranked by the Cross Encoder. This helps to find the best results if they were not found on the first try. As an early stopping mechanism against the possibility that the user set num hops to allow too many entries into the top K results, the algorithm will quit the process early before the current best top N candidate has a chance to be given to the top K results if it does not meet a certain cutoff (changeable parameter) threshold.

Model Training and Methodology

Our fake news detection system leverages pre-trained transformer models without additional fine-tuning. We employ Google's FLAN-T5-base (250M parameters) for both claim extraction and fact verification, utilizing its instruction-following capabilities with a temperature of 0.3 for deterministic outputs. The retrieval pipeline uses sentence-transformers/all-MiniLM-L6-v2 for embedding generation and cross-encoder/ms-marco-MiniLM-L6-v2 for reranking. Key hyperparameters include: TOP_X=3 clusters searched per query, TOP_N=10 candidates retrieved per hop, TOP_K=4 final reranked results, NUM_HOPS=2 for multi-hop retrieval, and a relevance CUTOFF=0.8 (80% similarity threshold). The evidence database undergoes PCA dimensionality reduction retaining 90% variance, followed by k-means clustering with k-means++ initialization into NUM_CLUSTERS=17 groups, balancing retrieval speed and quality. All models run on GPU (CUDA) for efficient inference, with the complete pipeline requiring approximately 3-5 minutes on Google Colab's T4 GPU for processing 100 news articles and generating verification results.

Results

For analysis we determined whether the result of the classification of the fake news by the LLM was correct. For this we used the standard confusion matrix to look at the true positive, true negative, precision, recall, and F1 scores. To test the validity of our model implementation, we used a subset of the standard FEVER (Fact Extraction and Verification) dataset. We used a balanced dataset of 300 samples that have claims supported by the graph information and 300 samples that have claims refuted by the graph information. We found that in all test cases our recall was high. We assume this comes down to our model's instruction to assume that a claim is false unless there is sufficient evidence to overturn the decision. This means that claims that retrieved information that was impartial or unideal would lead to a

Challenges

We had several challenges working on this project. The implementation of the RAG structure was fine, but many issues arose when providing inputs to the graph database. For many inputs during testing we noticed that the knowledge database of the graph did not contain the context necessary for the extraction of the correct evidence piece. This was because our input was sometimes faulty, touching upon a topic our model had no prior information on. Likewise, our evidence embedding chunks were grouped either so that vital information was spread across too many other chunks or too many random and useless blocks of information were grouped with the correct information, making the job of the LLM much tougher.

Conclusion

Future Work

This model has its limitations. If given more time to expand our project, there would be several ideas that we could follow to improve the implementation of our design. The most limiting factor is that our model is narrowly focused. If we were to give it much more data from many more news sources then the model would have even more options to generate the best supporting evidence and give the best results to the user. This way the range of topics could be explored much more in depth. We are currently limited by the scope and focus of our dataset, but incorporating many more datasets or verified news articles, or factual statements online, then our model would have much better performance and accuracy when determining whether or not the input query is real or fake and why. Another topic to explore would be another representation of the word clusters. We could look at different graph implementations to see which provided the best results and why. For this project we stuck with kMeans++ clustering to form graphs, and we are not sure if this is the best option or simply an option and why.

Works Cited

1. 2018 – Yang et al., “TI-CNN: Convolutional Neural Networks for Fake News Detection”
2. 2021 – Nasir et al., “Fake News Detection: A Hybrid CNN–RNN Deep Model”
3. 2024 – Chang et al., “Graph Global Attention Network with Memory (GANM)”